

Analytics Copilot

Ask Questions About Your Data in Plain English

Ben Blake & Tina Nguyen

University of Missouri–Kansas City ▪ CS 5542: Big Data Analytics

April 2026

github.com/ben-blake/analytics-copilot

The Problem

Without Analytics Copilot

1. Business user has a data question
2. Submits a request to a data analyst
3. Waits **3–5 days** for a dashboard or report
4. Analyst writes SQL by hand, often iterates

Result: Slow decisions, analyst bottleneck, non-technical users locked out of their own data.

With Analytics Copilot

1. Business user types a question in **plain English**
2. AI pipeline generates and validates Snowflake SQL
3. Results, chart, and SQL returned in **~7 seconds**

Result: Self-service analytics — no SQL knowledge required.

“Which product categories generate the most revenue?”

“What is the average delivery time by state?”

System Overview — Three-Agent Pipeline



1 Schema Linker

Uses **Cortex Search** (semantic RAG) to find the relevant tables from 9 in the database. Automatically adds FK join partners so the generator always has complete schema context.

2 SQL Generator

Sends the filtered schema to **llama3.1-70b** via a 15-rule structured prompt and a few-shot example bank. Produces Snowflake-compatible SQL with fully qualified table names.

3 Validator

Runs EXPLAIN on the generated SQL. If it fails, feeds the error back to the Generator for **up to 3 self-correction retries** before returning an error to the user.

Full System Architecture

Data Layer

Olist CSVs (9 tables, ~100k orders) ingested into ANALYTICS_COPILOT.RAW via Snowpark. Cortex LLM generates column descriptions and synonyms, stored in METADATA.TABLE_DESCRIPTIONS and indexed by a **Cortex Search Service**.

Agent Reasoning Layer

Schema Linker → SQL Generator → Validator run in sequence. A **PipelineTrace** module records per-agent timing and retry events, feeding both the Chat UI and the Monitor dashboard.

Domain Adaptation

CodeLlama-7B-Instruct fine-tuned with **LoRA** (rank 16, NF4 QLoRA) on 82 Alpaca-format Olist examples. Served via FastAPI as an alternative SQL generation backend.

Application Layer

Streamlit app with two tabs: **Chat** (query → SQL → table → auto-chart) and **Monitor** (session latency, success rate, agent step breakdown). Deployed to Streamlit Community Cloud.

Demo — Live Application

Chat Tab

User asks:

“Which states have the longest average delivery times?”

System does:

1. Schema Linker retrieves ORDERS + CUSTOMERS
2. SQL Generator writes a DATEDIFF-based query
3. Validator runs EXPLAIN, confirms valid, executes

Monitor Tab

- Total queries & success rate
- Average end-to-end latency
- Per-query history table
- Agent step breakdown chart (identify bottlenecks)

Deployment

- **Live:**
cs5542-analytics-copilot.streamlit.app
- Credentials via `st.secrets` (Streamlit Cloud)

Key Results

100%

Execution accuracy
on 50 golden queries

10%

Faster inference
LoRA vs. zero-shot baseline

82 examples

Sufficient for full
domain adaptation

Conclusion

What We Delivered

- **3-agent Text-to-SQL pipeline** — Schema Linker, SQL Generator, Validator — running on Snowflake Cortex
- **100% accuracy** on 50 golden queries (easy, medium, hard)
- **LoRA domain adaptation** — CodeLlama-7B, 82 examples, 10% speedup
- **Streamlit app** deployed to Community Cloud with Monitor dashboard

Future Work

- Multi-dataset support (any Snowflake schema)
- User feedback loop to improve golden query bank
- Voice input and richer chart types
- Fine-tune on Spider/BIRD benchmark for cross-domain transfer
- Enterprise multi-tenant deployment